

Access Modifiers in Java

Last Updated : 15 Dec, 2024



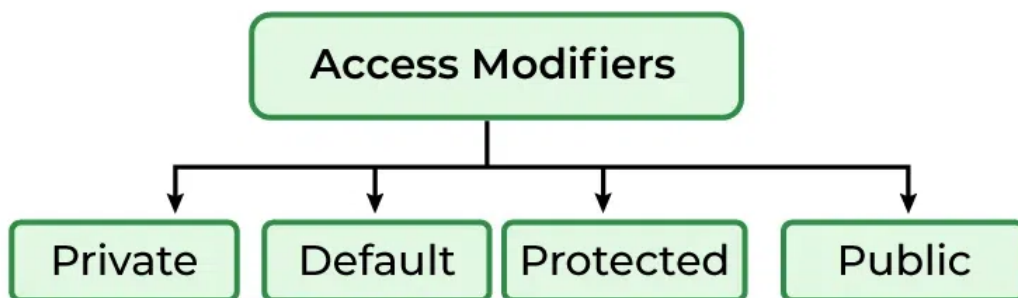
In Java, Access modifiers helps to **restrict the scope** of a class, constructor, variable, method, or data member. It provides **security, accessibility**, etc. to the user depending upon the access modifier used with the element. In this article, let us learn about **Java Access Modifiers**, their types, and the uses of access modifiers.

Types of Access Modifiers

There are **4 types of access modifiers** available in Java:

1. Default – No keyword required
2. Private
3. Protected
4. Public

Access Modifiers in Java



1. Default Access Modifier

When no access modifier is specified for a class, method, or data member, it is said to be having the **default** access modifier by default. The default access modifiers are **accessible only within the same package**.

Example 1: Demonstrating **Default Access Modifier Within the Same Package**. In this example, we will create two packages and the classes in the packages will be

having the default access modifiers and we will try to access a class from one package from a class of the second package.

Java



```
1 // default access modifier
2 package p1;
3
4 // Class Geek is having
5 // Default access modifier
6 class Geek
7 {
8     void display()
9     {
10         System.out.println("Hello World!");
11     }
12 }
```

Example 2: Error when **Accessing Default Modifier Class** across Packages. In this example, the program will show the **compile-time error** when we try to access a default modifier class from a different package.

Java



```
1 // error while using class from different
2 // package with default modifier
3 package p2;
4 import p1.*;    // importing package p1
5
6 // This class is having
7 // default access modifier
8 class GeekNew {
9     public static void main(String args[]) {
10
11         // Accessing class Geek from package p1
12         Geek o = new Geek();
13
14         o.display();
15     }
16 }
```

```
15     }
16 }
```

2. Private Access Modifier

The **private access modifier** is specified using the keyword **private**. The methods or data members declared as private are **accessible *only within the class in which they are declared***.

- Any other class of the **same package** will not be able to access these members.
- **Top-level classes or interfaces** can not be declared as private because,
 - **private** means “**only visible within the enclosing class**”.
 - **protected** means “**only visible within the enclosing class and any subclasses**”.

Hence these modifiers in terms of application to classes, apply only to [nested classes](#) and not on top-level classes.

Example: In this example, we will create two classes A and B within the same package p1. We will declare a method in class A as private and try to access this method from class B and see the result.

Java

```
1 // error while using class from different package with
2 // private access modifier
3 package p1;
4
5 // Class A
6 class A {
7     private void display() {
8         System.out.println("GeeksforGeeks");
9     }
10 }
11
12 // Class B
13 class B {
```

```

14         public static void main(String args[]) {
15             A obj = new A();
16
17             // Trying to access private method
18             // of another class
19             obj.display();
20         }
21     }

```

Explanation: The above code will show a compile-time error when trying to access a private method from class B, even within the same package.

3. Protected Access Modifier

The **protected access modifier** is specified using the keyword **protected**. The methods or data members declared as protected are *accessible within the same package or subclasses in different packages*.

Example 1: In this example, we will create two packages p1 and p2. Class A in p1 is made public, to access it in p2. The method display in class A is protected and class B is inherited from class A and this protected method is then accessed by creating an object of class B.

Java

```

1 // protected access modifier
2 package p1;
3
4 // Class A
5 public class A {
6     protected void display() {
7         System.out.println("GeeksforGeeks");
8     }
9 }

```

So, it demonstrates that a protected method is accessible within the same package.

Example 2: In this example, we will create two packages, p1 and p2. Class A in p1 has a protected method display. Class B in p2 extends A and accesses the protected method through [inheritance](#) by creating an object of class B.

Java

```
1 // protected modifier
2 package p2;
3
4 // importing all classes
5 // in package p1
6 import p1.*;
7
8 // Class B is subclass of A
9 class B extends A {
10     public static void main(String args[]) {
11         B obj = new B();
12         obj.display();
13     }
14 }
```

Explanation: The above example demonstrates that a **protected method is accessible in a subclass from a different package using inheritance.**

4. Public Access Modifier

The **public access modifier** is specified using the keyword **public**.

- The public access modifier has the **widest scope** among all other access modifiers.
- Classes, methods, or data members that are declared as public are **accessible from everywhere** in the program. There is no restriction on the scope of public data members.

Example 1: Here, the code shows that a **public method is accessible within the same package.**

Java



```
1 // public modifier
2 package p1;
3
4 public class A {
5
6     public void display() {
7         System.out.println("GeeksforGeeks");
8     }
9 }
```

Example 2: Here, the example shows that a **public method** is *accessible across packages*.

Java



```
1 // public access modifier
2 package p2;
3
4 import p1.*;
5
6 class B {
7     public static void main(String args[]) {
8
9         A obj = new A();
10        obj.display();
11    }
12 }
```

Important Points:

- If other programmers use your class, try to use the most restrictive access level that makes sense for a particular member. Use private unless you have a good reason not to.
- Avoid public fields except for constants.

Comparison Table of Access Modifiers in Java

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes

Algorithm to Use Access Modifier in Java

Here's a basic algorithm for using access modifiers in Java:

- **Define a class:** Create a class to represent the object you want to manage.
- **Define instance variables:** Inside the class, define variables for the data you want to manage.
- **Set an access modifier:**
 - Use **private** for variables *only accessible within the class*.
 - Use **protected** for variables *accessible within the class and its subclasses*.
 - Use **public** for variables *accessible from anywhere*.
- **Use getter and setter methods:** To access or modify variables, use **getter (accessor) and setter (mutator) methods**, even for public variables, to maintain encapsulation.

FAQs – Access Modifiers in Java

What are access modifiers in Java?

Access modifiers in Java are the keywords that are used for controlling the use of the methods, constructors, fields, and methods in a class.

What is the default access modifier in Java?

The default access modifier allows access within the same package only.

What are the 12 modifiers in Java?

*12 Modifiers in Java are public, private, protected, default, static, final, **synchronized**, abstract, **native**, **strictfp**, **transient**, and **volatile**.*

Can a private method be accessed outside its class?

No, a private method is only accessible within its own class.

What does the protected access modifier do?

The protected modifier allows access within the same package and by subclasses.

Can public variables be accessed from any class?

Yes, public variables are accessible from any class, regardless of the package.